

알고리즘 및 실습 최종 보고서 3조

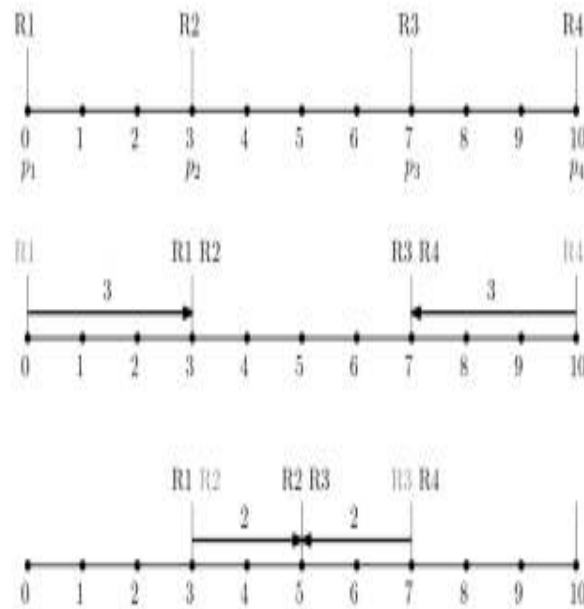
C189012 김민혁, C189018 김호준, C089038 오현준

I. 선택한 문제

Problem J Street Development Time Limit: 1.0 Seconds

ICPC street is currently an undeveloped area, with a large-scale development plan scheduled soon. Before starting the development, information about n important points along the street will be collected using n remote-controlled robots, with each robot collecting information from one of these important points. Now, the goal is to combine all the collected information into a single robot to find the most efficient development approach. To combine the information, the robots can move towards left or right and combine the information that they have from other robots. Also, each robot is powered by its own battery, and all the robots are equipped with identical batteries. Specifically, let $p_1, p_2, \dots, p_n$ represent the positions of the important points where the robots collect information, arranged from left to right. Then the requirements are as follows:

1. The ICPC street is considered as a one-dimensional interval $[0, L]$ with a positive integer L . The important points $p_1, p_2, \dots, p_n$ are always represented as integers on the interval, including two endpoints of the interval. That is, $p_1 = 0$ and $p_n = L$. Initially, each robot is positioned at one of the important points, so it has the information of the important point before beginning to move. Note that there is exactly one robot at each of these points initially, which means n is also the number of robots, and always at least 2 and at most $L + 1$.
2. For combining the information from other robots, robots can move freely to the left or right, consuming 1 unit of battery for 1 unit of distance traveled, regardless of direction. All robots are equipped with the same battery capacity with integer P , and move only in integer units of distance.
3. When two or more robots meet at the integer position on the street, they can combine each other's information. For example, if a robot with information about p_1 and p_2 encounters with a robot with information about p_3 and p_4 , both robots will then have information about the positions p_1, p_2, p_3 , and p_4 .
4. Robots consume the battery only for movement. Therefore, they do not use the battery when changing direction or when combining the information from other robots.
5. After all movements, at least one robot must have information about all the positions $p_1, p_2, \dots, p_n$.



For example, the figure above shows an example with $L = 10$, $n = 4$, and Robots 1, 2, 3, and 4 (R1, R2, R3, R4 in the figure) collect information (and are initially positioned) at $p_1 = 0$, $p_2 = 3$, $p_3 = 7$, and $p_4 = 10$, respectively. Then the following sequence of steps can be performed with a battery capacity of $P = 3$:

1. Robot 1 moves to p_2 , and Robots 1 and 2 combine each other's information.
2. Robot 4 moves to p_3 , and Robots 3 and 4 combine each other's information.
3. Robot 2 moves 2 units to the right, Robot 3 moves 2 units to the left, and they combine each other's information at the position 5 on the street.

Then after completing the process, Robots 2 and 3 will have information about all the positions p_1 , p_2 , p_3 , and p_4 .

Since the battery is much more expensive than the other parts of robot, it is important to determine the minimum battery capacity required for each robot for efficient data collection. Given L , n , and the positions of the important points p_1 , p_2 , ..., p_n , write a program to calculate the minimum battery capacity P required for at least one robot to have information about all the points.

* 2024 아시아-서울지역대회 Problem J (Street Development)

대규모 개발이 예정된 거리에 N 개의 중요 지점이 있으며, N 대의 원격 조종 로봇을 통해 각 지점의 정보를 수집해야 함. 이때 각 로봇은 하나의 중요 지점에서만 정보를 수집. 목표는 수집된 모든 정보를 단일 로봇에 통합하여 가장 효율적인 개발 방식을 찾는 것.

* 조건

- 개발 예정 거리는 0부터 L 까지의 1차원 구간이며, 양 끝점도 포함.
- N 개의 중요 지점에 각각 로봇 한 대씩 배치하며, 초기에는 해당 지점의 정보만 보유.
- 로봇은 좌우로 이동할 수 있으며, 이동 거리 1당 배터리 1만큼 소모.
- 모든 로봇은 동일한 배터리 용량을 가짐.
- 로봇들이 만나면 정보를 서로 교환할 수 있음.
- 방향 전환 및 정보 교환은 배터리를 소모하지 않음.
- 최종적으로 최소 1대의 로봇이 모든 지점의 정보를 보유해야 함.

* 최종 목표

- 모든 정보를 하나의 로봇에 통합하는 데 필요한 최소 배터리 용량.

* Input (L = 거리 , n = 로봇의 개수)

- 첫째 줄에 두 정수 L (거리 구간의 끝 위치)와 N (중요 지점(로봇) 개수)가 주어짐 ($1 \leq L \leq 10^6$, $2 \leq N \leq L+1$).
- 둘째 줄에는 N 개의 정수가 오름차순으로 주어지며, 각 정수는 로봇이 시작한 중요 지점의 위치임.

* Output

- 최소한 하나의 로봇이 모든 중요한 지점의 정보를 얻기 위해 필요한 최소 배터리 용량 P를 한 줄에 출력.

* Scenario sample

Sample Input 1	Output for the Sample Input 1
10 4 0 3 7 10	3
Sample Input 2	Output for the Sample Input 2
100 5 0 97 98 99 100	49
Sample Input 3	Output for the Sample Input 3
1 2 0 1	1

II. 알고리즘 설계

1) 가능한 배터리 범위 내에서 **이분 탐색**을 수행하며, 각 배터리 값에 대해 그리디 기반의 연결 가능성 검사를 진행.

2) 우선 가장 왼쪽 로봇부터 시작하여, 체인을 확장하듯 오른쪽으로 도달 가능한 최대 거리를 계산. 각 로봇이 앞선 로봇의 도달 범위 내에 존재하고, 배터리 소모량이 허용된다면 다음 로봇까지 연결을 이어감.

이와 동일한 방식으로, 오른쪽 끝 로봇부터 왼쪽 방향의 연결 가능성도 확인합니다. 이처럼 단방향 확산으로 전체 연결이 이루어진 경우, 해당 배터리값은 유효한 해로 취급.

3) 하지만 **그리디** 방식만으로는 항상 최적해를 보장하지 않기 때문에, 이를 위해 중간 허브 노드를 중심으로 좌우 체인을 간접적으로 연결할 수 있는지 추가로 검사. 허브 로봇이 좌우 체인 간의 거리를 충분히 커버할 수 있는 경우, 해당 배터리값은 연결 가능으로 판단됨. 이를 반복하여 최소한의 배터리값을 출력하도록 함.

III. 알고리즘 구현 설명

```
1  def can_connect_with_battery(positions, battery):
2      # 왼쪽 체인
3      left_pref = [None] * n
4      left_pref[0] = positions[0] + battery
5
6      for i in range(1, n):
7          prev = left_pref[i-1]
8          if prev is None:
9              break
10         cost = abs(positions[i] - prev)
11         if cost > battery:
12             break
13         new_reach = prev + (battery - cost)
14         left_pref[i] = new_reach
15         if new_reach >= positions[-1]:
16             return True
```

- 특정 배터리 용량으로 모든 로봇들이 서로 연결될 수 있는지를 확인하는 함수를 짜고자 함.
- 처음에 0에 위치한 즉 가장 왼쪽 지점에 있는 로봇은 배터리용량만큼 오른쪽으로만 갈수 있기에 position[0] + battery 로 초기화.
(None의 표기 이유 : 모든 로봇들이 연결이 되지 않았거나 애초에 불가능한 상황임을 표현하고자 함.)
- For 문 해석 : ① 왼쪽에서부터 오른쪽으로 하나씩 순차적으로 연결 시도
② 현재의 로봇이 이전 로봇이 도달한 최대 지점을 도달가능한지 체크.
③ 가능하다면 두 로봇이 연결됨을 확인함
-> 다음 로봇까지 도달 가능한 지점을 계산


```

17
18     # 오른쪽 체인
19     right_suff = [None] * n
20     right_suff[-1] = positions[-1] - battery
21
22     for j in range(n-2, -1, -1):
23         nxt = right_suff[j+1]
24         if nxt is None:
25             break
26         cost = abs(positions[j] - nxt)
27         if cost > battery:
28             break
29         new_reach = nxt - (battery - cost)
30         right_suff[j] = new_reach
31         if new_reach <= positions[0]:
32             return True

```

- 오른쪽에서 출발하는 로봇의 함수를 구현
- 시작점 : 지점의 배열 중 마지막 원소에서 배터리만큼 빼줌
- nxt에서 position[j]까지 접근 가능한지 보고 배터리가 남는다면 더 왼쪽으로 갈 수 있음을 의미하기 때문에 -로 방향 계산

```

33
34     # 허브 코드
35     for k in range(n):
36         L = positions[0] + battery if k == 0 else left_pref[k-1]
37         R = positions[-1] - battery if k == n-1 else right_suff[k+1]
38         if L is None or R is None:
39             continue
40         # 두 체인이 이미 만났는지
41         if L >= R:
42             return True
43         # 허브 k를 통해 순차 방문
44         span = R - L
45         dist_to_hub = min(abs(positions[k] - L),
46                           abs(positions[k] - R))
47         if span + dist_to_hub <= battery:
48             return True
49
50     return False
51

```

- 왼쪽 체인과 오른쪽 체인이 만나지 않는다면 허브를 중심 삼아 연결 시도
- 로봇 k를 허브로 설정 -> 양쪽 체인이 허브를 통해 연결이 가능한지 체크
- L= 왼쪽 체인이 최대한 도달 가능한 위치

- R= 오른쪽 체인이 도달 가능한 최대한의 위치
- $L \geq R$ 이라면, 두 체인이 겹치는 것을 의미하기에 중간 허브 없이도 이미 연결가능한 상태이므로 True를 반환.
- 두 체인이 겹치지 않은 경우 두 체인의 차와 허브가 이 경로에 접근하는데 필요한 최소거리가 배터리 용량 이하라면 연결이 가능함.
- 만약 이 경우에 포함이 안되는 경우라면, 그 배터리용량으로는 모든 로봇이 거리 전체를 커버할 수 없음.

```

52
53 def find_minimum_battery(positions):
54
55     lo, hi = 0, positions[-1] - positions[0]
56
57     while lo < hi:
58         mid = (lo + hi) // 2
59         if can_connect_with_battery(positions, mid):
60             hi = mid
61         else:
62             lo = mid + 1
63     return lo
64
65 if __name__ == "__main__":
66     import sys
67     input = sys.stdin.readline

```

- 이분 탐색으로 배터리의 최소값을 찾고자 함
- 배터리는 0이상이며 로봇 간 최대 거리 이상일 필요가 없으므로 lo, hi를 각각 0 과 거리의 최대치로 초기화 시켜줌.


```
68
69     L, n = map(int, input().split())
70     positions = list(map(int, input().split()))
71
72     print(find_minimum_battery(positions))
```

- 거리 길이를 L로 로봇의 수를 n으로 입력받고 각 로봇들이 위치할 지점들의 위치를 position 리스트에 저장시켜준다.
- 문제 상 input 조건에 증가하는 순서로 주어진다고 즉 정렬되어있는 채로 입력 받는다고 명시되어있기에 리스트 내 원소를 정렬할 필요가 없음을 확인함.

IV. 실행 결과

Sample ①

```
10 4
0 3 7 10
3
```

Sample ②

```
100 5
0 97 98 99 100
49
```

Sample ③

```
1 2
0 1
1
```

V. 알고리즘 시간복잡도 분석

```
def can_connect_with_battery(positions, battery):
    # 1) 왼쪽에서 시작하는 prefix 계산 (조기 종료 포함)
    left_pref = [None] * n
    left_pref[0] = positions[0] + battery

    for i in range(1, n):
        prev = left_pref[i-1]
        if prev is None:
            break
        cost = abs(positions[i] - prev)
        if cost > battery:
            break
        new_reach = prev + (battery - cost)
        left_pref[i] = new_reach
        if new_reach >= positions[-1]:
            return True

    # 2) 오른쪽에서 시작하는 suffix 계산 (조기 종료 포함)
    right_suff = [None] * n
    right_suff[-1] = positions[-1] - battery

    for j in range(n-2, -1, -1):
        nxt = right_suff[j+1]
        if nxt is None:
            break
        cost = abs(positions[j] - nxt)
        if cost > battery:
            break

    # 3) prefix/suffix 만으로 안 되면, 허브(k) 검사
    for k in range(n):
        L = positions[0] + battery
        R = positions[-1] - battery
        if L is None or R is None:
            continue
        # (1) 두 체인이 이미 만났는지
        if L >= R:
            return True
        # (2) 허브 k를 통해 순차 방문
        span = R - L
        dist_to_hub = min(abs(positions[k] - L),
                          abs(positions[k] - R))
        if span + dist_to_hub <= battery:
            return True

    return False

def find_minimum_battery(positions):
    """
    positions: 로봇 위치 리스트 (정렬되지 않아도 됨)
    return: 모든 로봇이 정보를 교환하기 위한 최소 배터리 B
    """
    lo, hi = 0, positions[-1] - positions[0]
    # lower_bound 스타일 이분 탐색
    while lo < hi:
        mid = (lo + hi) // 2
        if can_connect_with_battery(positions, mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

if __name__ == "__main__":
    import sys
    input = sys.stdin.readline
```

$O(n)$

```

def can_connect_with_battery(positions, battery):
    # 1) 왼쪽에서 시작하는 prefix 계산 (포기하고 포함)
    left_pref = [None] * n
    left_pref[0] = positions[0] + battery

    for i in range(1, n):
        prev = left_pref[i-1]
        if prev is None:
            break
        cost = abs(positions[i] - prev)
        if cost > battery:
            break
        new_reach = prev + (battery - cost)
        left_pref[i] = new_reach
        if new_reach >= positions[-1]:
            return True

    # 2) 오른쪽에서 시작하는 suffix 계산 (포기하고 포함)
    right_suff = [None] * n
    right_suff[-1] = positions[-1] - battery

    for j in range(n-2, -1, -1):
        nxt = right_suff[j+1]
        if nxt is None:
            break
        cost = abs(positions[j] - nxt)
        if cost > battery:
            break

    # 3) prefix/suffix 간격의 반, 허브(k) 찾기
    for k in range(n):
        L = positions[0] + battery
        R = positions[-1] - battery
        if L is None or R is None:
            continue
        # (1) 두 지점의 이미 연결되지
        if L >= R:
            return True
        # (2) 허브 k를 통해 숫자 범위
        span = R - L
        dist_to_hub = min(abs(positions[k] - L),
                          abs(positions[k] - R))
        if span + dist_to_hub <= battery:
            return True

    return False

```

$O(n)$

```

def find_minimum_battery(positions):
    """
    positions: 로봇 위치 리스트 (정렬되지 않아도 됨)
    return: 모든 로봇이 정보를 교환하기 위한 최소 배터리 B
    """
    lo, hi = 0, positions[-1] - positions[0]

    while lo < hi:
        mid = (lo + hi) // 2
        if can_connect_with_battery(positions, mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

if __name__ == "__main__":
    import sys
    input = sys.stdin.readline

```

$O(n \log L)$

최종 시간복잡도 : $O(n \log L)$